



ORDER-TO-CASH: Graph-Based Data Modelling & Query System — Technical Design Doc (v1.0)

Metadata

- Version: v1.0
- Date: March 2026
- Project: O2C Graph Query System
- Stack: Python · FastAPI · SQLite · React · Groq (Llama 3 70B)
- Deadline: 26 March 2026, 11:59 PM IST
- Status: In Progress — Active Build

Table of contents

1. Project Overview
2. System Architecture
3. Data Layer — Module A (ETL)
4. Cognitive Inference Module — Module B
5. Visual Intelligence Module — Module C
6. Fault Tolerance & Observability
7. Example Query Coverage
8. Build Plan & Progress Checklist
9. Architectural Decision Log
10. Project File Structure

1. Project Overview

1.1 Problem Statement

Real-world enterprise (ERP/SAP) data is fragmented across siloed tables — Orders, Deliveries, Invoices, and Payments — with no intuitive way to visualise or trace relationships. Business users cannot answer questions like “Why is this invoice delayed?” without manually joining multiple tables.

This project solves that by unifying fragmented ERP data into a queryable knowledge graph, exposed through a natural-language chat interface and an interactive graph visualisation.

1.2 Objectives

- Ingest and normalise raw business CSV data into a structured SQLite database
- Model the data as a property graph (nodes + edges) with an API abstraction layer
- Build a natural-language-to-SQL pipeline powered by an LLM (Groq / Llama 3 70B)
- Render an interactive force-directed graph explorer (react-force-graph)
- Implement domain-scoped guardrails so the system only answers O2C questions
- Deploy as a publicly accessible demo with no authentication required

1.3 Core Design Principles

- Think Graph, Store SQL: Graph model for conceptual clarity; SQLite for storage. LLMs generate SQL reliably — 10× more so than Cypher.
- Decoupling: ETL, Query Engine, and Visualisation are independent modules. LLM failure does not break the graph; graph failure does not break the chat.
- Grounding: Every AI response must trace back to an executed SQL query. No hallucination of data.
- Constraint-First: The LLM is given a read-only DB connection. It cannot INSERT, UPDATE, or DELETE.

- Observability: Every pipeline step is logged: intent classification result, generated SQL, execution time, row count returned.

2. System Architecture

Diagrams

Diagram 1 — System architecture (structural overview)

```
graph TD
    UA["User"] -->|"NL query"| FE["Frontend (Next.js)"]
    FE -->|"POST /api/query"| API["FastAPI backend"]

    subgraph A["Module A – Data orchestration (ETL)"]
        RAW["Raw CSVs"] --> PANDAS["Pandas clean + normalize"]
        PANDAS --> SQLITE["SQLite (entity tables + graph tables)"]
    end

    subgraph B["Module B – Cognitive inference"]
        GR["Guardrails\nTier-1 blocklist + Tier-2 LLM YES/NO"] --> PB["Prompt builder\nSchema + join map + few-shot"]
        PB --> LLM1["LLM → SQL gen\n(Groq · Llama 3 70B)"]
        LLM1 --> VAL["SQL validator\nSELECT-only + retry on failure"]
        VAL --> EXEC["Execute on SQLite (read-only)\n5s timeout"]
        EXEC --> SUM["LLM → summarise rows"]
    end

    subgraph C["Module C – Visual intelligence"]
        GV["Graph viewer\nreact-force-graph"]
        NI["Node inspector\nmetadata side panel"]
        CP["Chat panel\nanswer + SQL shown"]
        HL["Node highlight\nIDs from response"]
    end

    API -->|"GET /api/graph"| GV
```

```

API -->|"GET /api/graph/node/:id"| NI
API -->|"GET /api/graph/neighbors/:id"| GV
FE --> CP
CP -->|"highlight"| HL
HL --> GV

OBS["Observability layer\nstructured logs at each step"] --> API
SQLITE --> API
A --> SQLITE

```

Diagram 2 — End-to-end request flow (what happens on every query)

```

flowchart TD
    U["User types NL query"] --> FE["React chat panel\nPOST /api/query"]
    FE --> T1["Tier-1 guardrail\nVerb blacklist (instant)"]
    T1 -->|"blocked"| R1["Reject: Instant"]
    T1 -->|"pass"| T2["Tier-2 guardrail\nLLM binary YES/NO (~50 tokens)"]
    T2 -->|"off-topic"| R2["Reject: Off-topic"]
    T2 -->|"on-topic"| PB["Prompt builder\nSchema + join map + few-shot"]
    PB --> L1["LLM call 1\nSQL generation"]
    L1 --> V["SQL validator\nsqlparse + SELECT-only\nretry once"]
    V --> X["Execute on SQLite (read-only)\n5s timeout"]
    X -->|"empty result"| E["Return: No matching records\n(no LLM summarise)"]
    X -->|"rows"| L2["LLM call 2\nSummarise rows → answer"]
    L2 --> RESP["Response to frontend\nanswer + SQL + highlighted IDs"]
    E --> RESP

```

2.1 High-Level Architecture Overview

The system is divided into three primary modules plus a cross-cutting Observability layer:

- Module A — Data Orchestration (ETL): Ingest raw CSVs → clean → normalise IDs → load into SQLite entity tables + graph adjacency tables.
- Module B — Cognitive Inference (LLM): Accept natural language → guardrail check → build prompt with schema → LLM generates SQL → validate → execute → summarise answer.
- Module C — Visual Intelligence (Frontend): Render interactive force graph. Node expand, metadata inspector, chat panel, node highlighting from LLM responses.
- Cross-cutting — Observability Layer: Log every pipeline step with timestamps, inputs, outputs, and errors. Enables debugging and audit.

2.2 Request Flow — End to End

1. User types query in the React chat panel (e.g., "Which orders have no delivery?")
2. Frontend POSTs to `/api/query` on the FastAPI backend
3. Tier-1 guardrail: verb blocklist check (poem, write, tell me a story...). Instant reject if matched
4. Tier-2 guardrail: LLM binary check — "Is this about O2C business data? YES/NO". Reject if NO
5. Prompt builder injects full SQLite schema + join paths + 3–4 few-shot examples
6. LLM (Groq / Llama 3 70B) generates a SELECT query
7. SQL validator parses the query, checks it is SELECT-only. If invalid, retry once
8. Query executes against SQLite via the read-only DB connection. Timeout: 5 seconds
9. If result is empty → return "No matching records found." — LLM does NOT speculate
10. LLM summarises rows into a natural-language answer (second LLM call)
11. Response returned to frontend: answer text + the generated SQL (shown in a collapsible panel)

12. Frontend parses any entity IDs in the response and highlights those nodes in the graph

2.3 Technology Stack

- Backend framework: FastAPI (Python) — async, type-safe, fast to build, excellent for AI pipelines
- Primary database: SQLite — zero-ops, sufficient for dataset size, LLMs generate SQL reliably
- LLM provider: Groq (Llama 3 70B) — free tier, high token/s, low latency for demo
- LLM client: Direct Groq API (no LangChain) — fewer abstractions, easier to debug, full prompt control
- Data processing: Pandas — standard for CSV normalisation and ETL in Python
- Frontend framework: Next.js (React) — component model fits graph + chat split layout well
- Graph visualisation: react-force-graph-2D (WebGL/D3) — handles hundreds of nodes smoothly
- Styling: Tailwind CSS — utility-first, fast to build professional-looking UI
- Deployment: Railway or Render — single-command deploy, free tier, no infra setup

3. Data Layer — Module A (ETL)

Diagrams

Diagram 3 — Data / graph model (entity relationships)

```
graph TD
    C["Customer"] -->|"PLACED_BY"| SO["Sales order"]
    SO -->|"CONTAINS"| OI["Order item"]
    OI -->|"IS_MATERIAL"| M["Material"]

    SO -->|"FULFILLED_BY"| D["Delivery"]
```

```
D -->|"SHIPS_FROM"| P["Plant"]
D -->|"SHIPS_TO"| A["Address"]

BD["Billing doc"] -->|"BILLS"| D
BD -->|"BILLED_TO"| C
PAY["Payment"] -->|"SETTLES"| BD

JE["Journal entry"] -->|"LINKED_TO"| BD
```

3.1 Graph Model (Conceptual)

We use a Property Graph Model for conceptual clarity. Even though SQLite is the storage engine, every entity is a node and every foreign-key relationship is a directed edge. This model is what the API exposes to the frontend.

Node types (entity tables):

- Customer — who placed the order
- SalesOrder — the root entity of the O2C flow
- SalesOrderItem — individual line items within an order
- Material / Product — what was ordered
- Delivery — shipment fulfilling an order
- Plant — warehouse or facility where delivery originates
- Address — ship-to location for a delivery
- BillingDocument / Invoice — financial document raised against a delivery
- Payment — settlement of an invoice
- JournalEntry — accounting record linked to a billing document

Key relationships (edge types):

- (SalesOrder) -[CONTAINS]→ (SalesOrderItem)
- (SalesOrder) -[PLACED_BY]→ (Customer)
- (SalesOrderItem) -[IS_MATERIAL]→ (Material)
- (SalesOrder) -[FULFILLED_BY]→ (Delivery)
- (Delivery) -[SHIPS_FROM]→ (Plant)

- (Delivery) -[SHIPS_TO]→ (Address)
- (BillingDocument) -[BILLS]→ (Delivery)
- (BillingDocument) -[BILLED_TO]→ (Customer)
- (Payment) -[SETTLES]→ (BillingDocument)
- (JournalEntry) -[LINKED_TO]→ (BillingDocument)

3.2 SQLite Schema

```

sales_orders      (order_id TEXT PK, customer_id FK, order_
date, status, total_amount)
sales_order_items (item_id TEXT PK, order_id FK, material_i
d FK, qty, unit_price)
customers         (customer_id TEXT PK, name, region, segme
nt)
materials        (material_id TEXT PK, description, catego
ry, unit)
deliveries        (delivery_id TEXT PK, order_id FK, plant_
id FK, address_id FK, delivery_date, status)
plants           (plant_id TEXT PK, name, location)
addresses         (address_id TEXT PK, street, city, countr
y)
billing_docs      (billing_id TEXT PK, delivery_id FK, cust
omer_id FK, amount, due_date, status)
payments         (payment_id TEXT PK, billing_id FK, amoun
t, payment_date)
journal_entries   (journal_id TEXT PK, billing_id FK, compa
ny_code, fiscal_year, amount)

-- Graph adjacency (served to frontend)
nodes (node_id TEXT PK, node_type TEXT, label TEXT, proper
ties JSON)
edges (edge_id TEXT PK, source_id FK, target_id FK, relati
on_type TEXT)

```

3.3 ETL Pipeline — `ingest_data.py`

WHAT: A standalone Python script that reads raw CSV/JSON files, cleans and normalises them, and populates the SQLite database.

WHY: Keeping ETL isolated means if the source data changes, only this script needs updating. The rest of the system is unaffected.

WHERE: Runs once before the server starts. Re-runnable via a `/admin/reload` endpoint.

WHEN: Run manually on Day 1.

Steps performed:

1. Read source files with `pandas.read_csv()` / `read_json()`
 2. Normalise entity IDs (strip whitespace, enforce consistent casing, handle nulls)
 3. Deduplicate rows on primary keys
 4. Create SQLite tables with `CREATE TABLE IF NOT EXISTS`
 5. Create indexes on all foreign key columns (critical for JOIN performance)
 6. Populate entity tables from cleaned CSVs
 7. Populate graph adjacency tables (nodes + edges) from entity tables
 8. Write an `entities_index` table listing all IDs and types
-

4. Cognitive Inference Module — Module B

4.1 Guardrail System

WHAT: A two-tier filter that blocks off-topic queries before they consume LLM tokens.

WHY: Protects against misuse, reduces cost, satisfies the evaluation criterion on domain restriction.

WHERE: First code executed in the `/api/query` handler, before any LLM call.

WHEN: On every single user query, with no exceptions.

Tier mechanisms:

- Tier 1: Verb blocklist (e.g., "poem", "write a story", "translate", "generate code", "explain history") — zero cost
- Tier 2: LLM binary check for ambiguous queries — ~50 tokens

Tier-2 prompt template:

```
System: You are a classifier for an Order-to-Cash ERP system.
        Reply only with YES or NO.
User:   Is this query about O2C business data (orders, deliveries,
        invoices, payments, customers, materials)?
Query: "{user_query}"
```

4.2 Prompt Engineering — Text-to-SQL

WHAT: A structured system prompt injected with the full database schema, known join paths, and few-shot examples before every SQL generation call.

WHY: The LLM has no memory of the schema. Without schema injection, it will hallucinate table and column names. Few-shot examples teach it the exact join patterns.

WHERE: Assembled by `prompt_builder.py`. Cached at startup — schema does not change at runtime.

WHEN: On every query that passes guardrails, as the first of two LLM calls.

System prompt structure:

1. Role: "You are a SQL analyst for an O2C ERP system."
2. Schema: Full CREATE TABLE statements with column names and types
3. Join map: Explicit FK relationships (which column joins which table)
4. Rules: SELECT only. LIMIT 50. No subqueries joining >4 tables. Return UNABLE_TO_QUERY if not answerable. Return raw SQL only.
5. Examples: 3–4 NL → SQL pairs

Few-shot examples:

```
Q: Which orders have no delivery?
A: SELECT o.order_id, o.order_date
   FROM sales_orders o
   LEFT JOIN deliveries d ON o.order_id = d.order_id
   WHERE d.delivery_id IS NULL;
```

Q: Trace billing document **BILL-001** back to the original order.

A: **SELECT** b.billing_id, d.delivery_id, o.order_id, o.order_date

```
FROM billing_docs b
JOIN deliveries d ON b.delivery_id = d.delivery_id
JOIN sales_orders o ON d.order_id = o.order_id
WHERE b.billing_id = 'BILL-001';
```

Q: Which products appear in the most billing documents?

A: **SELECT** m.description, **COUNT**(**DISTINCT** b.billing_id) **as** billing_count

```
FROM sales_order_items soi
JOIN materials m ON soi.material_id = m.material_id
JOIN sales_orders o ON soi.order_id = o.order_id
JOIN deliveries d ON o.order_id = d.order_id
JOIN billing_docs b ON d.delivery_id = b.delivery_id
GROUP BY m.material_id
ORDER BY billing_count DESC
LIMIT 10;
```

4.3 SQL Validation & Retry

WHAT: After the LLM returns SQL, a validation step parses it and checks constraints before execution.

WHY: LLMs occasionally produce invalid SQL or accidentally include a non-SELECT statement. Catching this before execution protects the database.

WHERE: `validator.py`, between LLM call 1 and DB execution.

WHEN: After every SQL generation attempt, with up to one retry.

Validation checks:

- Parse SQL with `sqlparse` — catch syntax errors
- Assert first token is SELECT — reject INSERT/UPDATE/DELETE/DROP
- Check referenced table names exist in known schema
- If validation fails: retry once with the error appended to the prompt

- If retry also fails: return "I was unable to generate a valid query for this question."

4.4 Answer Summarisation

WHAT: A second, lighter LLM call that converts raw SQL result rows into a readable natural-language answer.

WHY: Raw tabular data is not user-friendly.

WHERE: After successful query execution, before returning the response to the frontend.

WHEN: Only if the query returned at least one row. Empty results return a canned message.

Empty result handling:

```
if len(rows) == 0:
    return "No matching records found in the dataset for your query."
    # LLM is NOT called. No speculation allowed.
```

5. Visual Intelligence Module — Module C

Diagrams

Diagram 4 — Frontend UI layout

```
flowchart LR
    subgraph W["02C Graph Explorer – browser window"]
        direction LR
        subgraph L["Graph viewer (left panel – 60%)"]
            direction TB
            G["Force-directed graph\n(nodes + edges)\nextended on click"]
            S["Search nodes..."]
        end
        subgraph R["Chat + inspector (right – 40%)"]
            direction TB
        end
    end
```

```

        Q["Chat thread\n• user question\n• AI answer\n•
View SQL (collapsible)"]
        IN["Ask a question... (input)"]
        NI["Node inspector\nid · type · properties\nshown on node click"]
    end
end
L --- R

```

5.1 Frontend Architecture

WHAT: A Next.js application with two primary panels — graph visualisation on the left, chat interface on the right.

WHY: The split layout mirrors the reference UI in the assignment PDF.

WHERE: `/frontend` directory.

WHEN: Always available, independent of the LLM. Graph browsing works even if the AI pipeline is down.

5.2 Graph Visualisation

Library: `react-force-graph-2D` (WebGL-backed D3 force simulation)

- Nodes colour-coded by entity type (Order = blue, Delivery = teal, Invoice = amber, Payment = green, Customer = purple)
- Directed edges with arrows labelled with relationship type
- Click node → `GET /api/graph/neighbors/{node_id}` and append connected nodes (expand pattern)
- Side panel shows node metadata when selected
- Nodes referenced in chat responses highlighted with a glowing ring animation
- Search box filters nodes by ID or label without a backend call

5.3 Chat Panel

- Scrollable message history
- Each AI response includes collapsible “View SQL” showing the generated query

- Entity IDs in responses are clickable and jump to the node in the graph
- Loading state during pipeline (typically 1–3 seconds on Groq)
- Error state on non-200 backend response

5.4 API Endpoints

- POST `/api/query` : Full NL pipeline — returns answer + SQL (Chat panel)
 - GET `/api/graph` : All nodes + edges (paginated) (Graph init)
 - GET `/api/graph/neighbors/{id}` : Direct neighbors (Node expand)
 - GET `/api/graph/node/{id}` : Full metadata (Inspector)
 - GET `/api/health` : Health check (Deployment monitoring)
-

6. Fault Tolerance & Observability

6.1 Blast Radius Analysis

- LLM hallucination: Query validation + empty-result canned response → incorrect text possible; DB untouched (Designed)
- Invalid SQL: sqlparse check + one retry → user sees “unable to query” (Designed)
- LLM API timeout/outage: 5-second timeout; fallback message → graph still works (Designed)
- Heavy query overload: 5-second execution timeout → query killed; DB performant (Designed)
- Data corruption by LLM: Read-only SQLite connection → no data loss possible (Designed)
- Off-topic prompt abuse: Two-tier guardrail → rejected before SQL generation (Designed)
- ETL script failure: Standalone ETL; app checks DB health on startup → clear error, refuses to start (Designed)
- Frontend graph render error: React Error Boundary → chat continues (Designed)

6.2 Observability Strategy

Every step in the pipeline emits a structured log entry with:

- timestamp (ISO 8601)
- query_id (UUID per request)
- step: GUARDRAIL_T1, GUARDRAIL_T2, PROMPT_BUILD, SQL_GEN, SQL_VALIDATE, SQL_EXEC, SUMMARISE, RESPONSE
- status: PASS / FAIL / REJECT / ERROR
- duration_ms
- payload: truncated input/output (SQL text, row count, error message)

Logs written to stdout (Railway/Render captures automatically) and optionally to a `logs.db` SQLite table.

7. Example Query Coverage

- Which products have the most billing documents? → GROUP BY material + COUNT billing_docs (Required)
- Trace full flow of billing doc X → billing → delivery → order (Required)
- Orders delivered but not billed → LEFT JOIN billing_docs WHERE billing_id IS NULL (Required)
- Orders billed without delivery → LEFT JOIN deliveries WHERE delivery_id IS NULL (Required)
- Total revenue per customer → SUM(billing_docs.amount) GROUP BY customer (Bonus)
- Average delivery time per plant → AVG(delivery_date - order_date) GROUP BY plant (Bonus)
- Overdue invoices not yet paid → WHERE due_date < today AND payment_id IS NULL (Bonus)
- "Write me a poem about invoices" → Rejected by guardrail (Guardrail demo)

8. Build Plan & Progress Checklist

Day 1 — Data Foundation (ETL)

- ☐ Download and inspect dataset (understand all columns and file formats)

- ☐ Create `ingest_data.py` with Pandas
- ☐ Define and create all SQLite tables
- ☐ Normalise entity IDs (strip whitespace, consistent casing, null handling)
- ☐ Create indexes on all foreign key columns
- ☐ Populate entity tables from cleaned CSVs
- ☐ Populate graph adjacency tables (nodes + edges)
- ☐ Create `entities_index` table (all IDs + types)
- ☐ Verify schema with manual SQL queries
- ☐ Commit ETL script to GitHub

Day 2 — Backend & LLM Pipeline

- ☐ Initialise FastAPI project structure
- ☐ Implement `/api/health`
- ☐ Implement `/api/graph` (all nodes + edges, paginated)
- ☐ Implement `/api/graph/neighbors/{id}`
- ☐ Implement `/api/graph/node/{id}`
- ☐ Build Tier-1 guardrail (verb blocklist)
- ☐ Build Tier-2 guardrail (LLM binary check via Groq)
- ☐ Build `prompt_builder.py` (schema injection + few-shot examples)
- ☐ Integrate Groq API for SQL generation
- ☐ Build SQL validator (sqlparse + SELECT-only check)
- ☐ Implement retry logic (one retry with error context)
- ☐ Execute query against read-only SQLite connection
- ☐ Implement empty-result canned response
- ☐ Build summarisation call (rows → NL answer)
- ☐ Implement full `/api/query` wiring
- ☐ Add structured logging to all pipeline steps
- ☐ Test all queries with curl

- ☐ Commit backend to GitHub

Day 3 — Frontend

- ☐ Initialise Next.js project with Tailwind CSS
- ☐ Create split layout: graph left, chat right
- ☐ Integrate react-force-graph-2D
- ☐ Fetch `/api/graph` on load and render initial graph
- ☐ Colour-code nodes by entity type
- ☐ Add directed edge labels
- ☐ Node click → expand neighbours
- ☐ Node inspector side panel
- ☐ Chat panel UI
- ☐ Wire chat to POST `/api/query`
- ☐ Loading state
- ☐ Error state
- ☐ Collapsible "View SQL"
- ☐ Highlight referenced nodes
- ☐ Node search / filter
- ☐ React Error Boundary around graph
- ☐ Commit frontend to GitHub

Day 4 — Polish, Deploy & Submit

- ☐ End-to-end test all 8 example queries
- ☐ Test guardrail cases (off-topic queries)
- ☐ Test empty result handling
- ☐ Test node expand on all entity types
- ☐ Fix UI layout issues
- ☐ Write README (architecture, stack choices, prompting, guardrails)
- ☐ Record 2-minute demo walkthrough video

- ☐ Deploy backend to Railway / Render
 - ☐ Deploy frontend to Vercel
 - ☐ Verify live demo URL is public
 - ☐ Export AI session transcripts
 - ☐ Submit via Google Form (demo link + GitHub repo)
-

9. Architectural Decision Log

ADL-01: SQLite over Neo4j

WHAT: Use SQLite as the primary data store for all entity data and graph adjacency.

WHY: LLMs generate SQL reliably (trained on vastly more SQL than Cypher). SQLite is zero-ops (single file), no cold-start latency, sufficient for dataset size. Neo4j's traversal advantage matters at millions of nodes.

WHERE: `ingest_data.py` (write), FastAPI handlers (read), `prompt_builder.py` (schema reference).

WHEN: Foundational decision. Revisit only if dataset exceeds 100K nodes.

Interview talking point: "I evaluated Neo4j but for this dataset size, SQLite + SQL generation is more reliable and faster to demo."

ADL-02: Direct Groq API over LangChain

WHAT: Call the Groq API directly using the openai-compatible client. No LangChain/LangGraph.

WHY: Avoid abstraction overhead; easier debugging; full prompt control.

WHERE: `cognitive/llm_client.py`

WHEN: Decided on Day 1.

Interview talking point: "My pipeline is two sequential LLM calls. Adding a framework would mean debugging the framework, not the problem."

ADL-03: Two-Tier Guardrail (Blocklist + LLM Check)

WHAT: Domain restriction implemented as two checks: verb blocklist + LLM binary classifier.

WHY: Keyword-only has false positives/negatives; pure LLM check is too slow/expensive for every query.

WHERE: `cognitive/guardrail.py`

WHEN: Every query, before any other processing.

ADL-04: Think Graph, Store SQL

WHAT: Model as a property graph but store/query using SQLite.

WHY: Graph model provides clarity and clean API for frontend; SQL is reliable for LLM generation and easy to operate.

WHERE: Throughout docs/README; storage in SQLite; API in FastAPI.

WHEN: Foundational decision.

ADL-05: Read-Only DB Connection for LLM Queries

WHAT: Open the SQLite connection in read-only mode.

```
conn = sqlite3.connect("file:data.db?mode=ro", uri=True)
```

WHY: Defence-in-depth — even if validator fails, the DB driver rejects writes.

WHERE: `cognitive/db_executor.py`

WHEN: Always.

10. Project File Structure

```
o2c-graph/
├── backend/
│   ├── main.py           # FastAPI app entry point
│   ├── ingest_data.py    # ETL: CSV → SQLite
│   └── cognitive/
│       ├── guardrail.py  # Two-tier guardrail
│       └── prompt_builder.py # Schema injection + few-shot
│   ├── llm_client.py     # Groq API wrapper
│   ├── validator.py      # SQL parse + SELECT-only
│   └── db_executor.py     # Read-only SQLite execut
```

```

ion
|   └─ graph/
|   └─ graph_api.py           # /api/graph endpoints
|   └─ data.db                # SQLite database (gitign
ored)
|   └─ requirements.txt
└─ frontend/
    └─ pages/
    └─ index.js               # Main split layout page
    └─ components/
    └─ GraphViewer.jsx        # react-force-graph wrapp
er
    └─ ChatPanel.jsx          # Chat interface
    └─ NodeInspector.jsx      # Metadata side panel
    └─ ErrorBoundary.jsx      # React error boundary
    └─ package.json
└─ data/                     # Raw source CSVs (gitign
ored)
└─ README.md
└─ .env.example              # GROQ_API_KEY placeholde
r

```

11. Submission Requirements Checklist

- ☐ Working demo link (publicly accessible, no auth required) — Pending deploy
- ☐ Public GitHub repository — Pending
- ☐ README: architecture decisions — Day 4
- ☐ README: database choice rationale (SQLite over Neo4j) — Day 4
- ☐ README: LLM prompting strategy (schema injection + few-shot) — Day 4
- ☐ README: guardrail implementation — Day 4
- ☐ AI coding session logs / transcripts — Export Day 4
- ☐ Graph construction with nodes and edges — Day 1-2
- ☐ Graph visualisation with expand + inspect — Day 3

- ☐ Conversational query interface (NL → SQL → answer) — Day 2–3
- ☐ Example query: products with most billing docs — Day 2
- ☐ Example query: full flow trace — Day 2
- ☐ Example query: broken / incomplete flows — Day 2
- ☐ Guardrails: reject off-topic queries — Day 2
- ☐ Bonus: show generated SQL in UI — Day 3
- ☐ Bonus: highlight referenced nodes in graph — Day 3